

TDD - Test Driven Development

¿Qué tiene de importante TDD?

- ★ Nos permite tener una detección de errores mucho más temprana, permitiéndonos enfocarnos en lo que tenemos que hacer.
- ★ Nos ayuda a prevenir irnos de mambo. Obliga a solo hacer lo necesario.

Como desarrollador, es necesario definir muy bien qué voy a hacer, porque el cliente NO PAGA por algo que NO PIDIÓ, y nosotros NO TRABAJAMOS gratis.

Las pruebas no se tocan a menos que haya un cambio en el negocio, implicando que lo que íbamos a probar originalmente ya no exista. NO cambio las pruebas si no pasan.

La prueba unitaria es atómica. Porque si dentro de la prueba hay 2 validaciones y una falla y la otra pasa, ¿qué termina siendo?

TP6

Se evalúa:

- ★ Uso de la técnica de US
- ★ TDD → usando el repo, vamos a programar esta US completa.
 - Pruebas Unitarias.
 - Uso del repositorio. → Pruebas primero, Código después.
- ★ Importa que el software esté funcionando.
- ★ Documento de Estilos de Código.
 - Podemos usar una guía ya hecha (por ejemplo: Pep8 para Python) o crear una guía propia.

Nos tocó la US: Comprar entradas

Comprar entradas	8
COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque	
Criterios de Aceptación: ● Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada visitante y tipo de pase (VIP o regular). ● La fecha de visita guiada puede ser del día actual o futuro. ● Debe enviar un mensaje de confirmación vía mail ● Debe redirigir a mercado pago al confirmar la compra si el pago es con tarjeta de crédito ● La fecha de la visita debe estar dentro de los días en que el parque está abierto. ● Debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta ● La cantidad de entradas requeridas no debe ser mayor a 10 ● Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha. ● Se debe permitir la compra de entradas solo a usuarios registrados.	
Pruebas de usuario: - Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de pago con tarjeta mediante Mercado pago y recepción del mail de confirmación (pasa) - Probar comprar entradas sin seleccionar forma de pago (falla) - Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra cerrado (falla) - Probar comprar entradas ingresando una cantidad de entradas mayor a 10 (falla)	

No se pide hacer conexión con MP, pero se debe simular esa comunicación. Existen algunas librerías como “Mock” o “Mockito” para hacerlo.

Lenguaje elegido: PYTHON

Guía de Estilos a seguir: PEP8

Lista de Comunicaciones a Resolver:

- Mercado pago ¿Tarjetas de crédito/débito?
- Mail

Lista de Pruebas:

-
-

DUDAS (a charlar con el cliente)

★ **Tipos de entradas:**

¿Desde cuándo se estableció la distinción entre entradas *VIP* y *Regulares*? ¿Podría especificar qué diferencias o beneficios implica cada tipo?

★ **Determinación del precio:**

¿El valor de las entradas será definido por nuestro equipo de desarrollo o lo establecerá usted? Esto es importante para poder calcular y mostrar correctamente el total a pagar.

★ **Autenticación de usuario:**

¿Desea que se implemente un sistema real de inicio de sesión (*login*) para esta etapa, o podemos simular el acceso del usuario únicamente con fines de prueba de esta User Story?

★ **Disponibilidad del parque:**

¿Podría indicar los días y horarios en que el parque permanece abierto o cerrado, a fin de reflejar correctamente las fechas disponibles para la compra de entradas?

★ **Medios de pago:**

¿Las compras podrán realizarse con tarjeta de débito, o únicamente con tarjeta de crédito a través de la pasarela de pago de *Mercado Pago*?